**GOOGLE | 1POINT3ACRES**

# Other rounds: system design, ML and behavioral

*9 non-coding rounds from the Google question bank, as the site describes them. Each starts on its own page.*

1. L6 System Design (Staff Loop) (system design, asked 2026-07-06)
2. Web Crawler at Search Scale (system design, asked 2026-08-16)
3. Large-Scale Interactive Map Visualization (system design, asked 2026-08-29)
4. ML System Design (Recsys / Chatbot / Image Classifier) (system design, asked 2026-10-02)
5. Behavioral Round (Googliness Themes) (behavioral, asked 2026-10-16)
6. CPU Architecture and Simulator Design Deep Dive (other round, asked 2026-07-24)
7. Research Paper Presentation and Defense (DeepMind RS) (other round, asked 2026-08-13)
8. Google Hiring Assessment (GHA / Work Styles) (other round, asked 2026-09-03)
9. ML Fundamentals Deep Dive (AI/ML & MLE Roles) (other round, asked 2026-09-12)

SYSTEM DESIGN | 60 min

# L6 System Design (Staff Loop)

Frequency: medium | Last asked: 2026-07-06 | Stage: onsite-system-design, onsite-deep-dive | Roles: SWE, INFRA-ENG, EM | Tags: system-design, scaling, storage, messaging, rate-limiting
Original: 1point3acres.com/.../l6-system-design-staff-loop

*L6 (Staff) onsite includes two System Design rounds (one infra-leaning, one product-leaning), each 60 min. Common prompts include distributed rate limiter, global notification system, anomaly detection service, Find-My-Device, dictionary range query store, and global chain restaurant menu sync. Senior bar is on requirements clarification, deep-dives into trade-offs, failure modes, and evolution path.*

## Requirements

### Common prompts (each is one 60-min round)

- Distributed rate limiter as an internal service (10■+ QPS, multi-tenant, multi-region).
- Quota limiter / Google Drive-style usage quota — clarify whether the requirement is request throttling, per-user/per-tenant resource quota, or storage-usage enforcement before choosing counters, windows, and reconciliation semantics.
- Global real-time notification system (push + email + SMS, 10■+ users, fan-out + retry + dedup).
- Anomaly detection system for an unspecified domain (clarify first: metric stream? user behavior? infra alerts?).
- Find-My-Device (Find My iPhone) — explicit privacy, security, efficiency requirements.
- Global chain restaurant menu update system — multi-region menu sync, offline-tolerant edge devices, time-of-day variants.
- Dictionary range query — store a set of words; support range(a, b) returning all words in lexicographic interval. Discuss trie + sorted index + sharding.

### Round structure (universal across prompts)

- Requirements clarification (~10 min) — explicitly ask about: internal vs external, peak QPS, latency SLO, precision/recall trade-offs, region geometry, consistency expectations, security model.
- High-level architecture (~10 min) — top-level boxes (gateway, service tier, storage, async pipeline). Resist diving into one box too early.
- Deep dive (~25 min) — interviewer picks one or two boxes and grills. Expect: hot key handling, cache invalidation, sliding window vs token bucket, hierarchical limiting, sharding scheme, leader election, write quorum.
- Failure & trade-off (~10 min) — what breaks when a region disconnects, when storage fails, when a tenant abuses the system. Concrete fail-open vs fail-close decisions.
- Evolution path (~5 min) — how does the system grow from MVP → 10× → 100×? Where do you re-architect? How do you roll out the new design?

## Notes

- L6 interviewers explicitly compare candidates against L5; the differentiator is decision-making depth + cross-team influence reasoning, not architecture novelty.

## Common L6 failure modes

- Jumping into architecture without clarifying requirements (the most cited pitfall).
- Architecture looks complex but candidate cannot defend each box's necessity.

- System design pitched at L5 ("here is a sharded cache") rather than L6 ("here is how I'd choose between sharded cache and Redis Cluster given the consistency requirements, and here is how I'd roll out the change").
- For the open-ended product prompt (anomaly detection, find-my-device), the first 10 minutes of clarification is graded — vague answers there poison the rest of the round.
- The L6 BQ round in the same loop is the second-largest signal: stories should center on cross-team influence, technical decision rationale, owning outages, and disagreeing with senior peers.

## Preparation

- Drill 6–8 distinct system design prompts cold (rate limiter, notification, search, chat, payment, ride-sharing, file storage, real-time analytics). For each prompt, prepare:
- 2 explicit trade-offs you'd present.
- 1 named failure case + mitigation.
- 1 evolution path (MVP $\rightarrow$ 10×).
- Build a personal cheatsheet of capacity math: 1M QPS budgets, 99p latency on Redis vs SQL vs Spanner, replication lag numbers, cost per TB on common storage tiers.
- Practice the requirements-clarification opener as a 3–5 question script; speak it out loud at the start of every mock.
- Read Designing Data-Intensive Applications chapters 1–9; do the exercises.
- For each loop, write down a 1-page Past Project Deep Dive (the 5th round) covering: design decision, what was wrong with it, what you'd do today — interviewers grade reflection here.

SYSTEM DESIGN | 45 min

# Web Crawler at Search Scale

Frequency: single | Last asked: 2026-08-16 | Stage: onsite-system-design | Roles: SWE | Tags: web-crawler, distributed-systems, scaling, dedup, hard
Original: 1point3acres.com/.../web-crawler-at-search-scale

*Design a periodic crawler for tens of billions of pages feeding a search index, without JavaScript rendering. Deep dives cover a two-level URL frontier, robots.txt caching, exact and near-duplicate detection, crawler traps, host-based sharding, checkpoint recovery, and a 24-hour full-recrawl target.*

## Requirements

- Crawl tens of billions of pages, revisit them periodically, and deliver parsed content to a search index. JavaScript rendering is out of scope.
- Close the loop as URL frontier → fetcher → parser → deduplication → storage, with newly extracted URLs fed back into the frontier.
- Balance priority with host politeness. Use front queues by priority and back queues by host so requests to one host are serialized and spaced while high-priority URLs still advance quickly. Define a robots.txt cache policy.
- Deduplicate both URLs and content. URL-level membership can combine hashing with a Bloom filter; content-level detection must handle the same page appearing at multiple URLs, using checksums for exact matches and a near-duplicate signal such as SimHash.
- Detect crawler traps such as infinite calendar pages and dynamically generated URL spaces by limiting per-domain depth and volume and recognizing pathological URL patterns.
- Distribute work by consistently hashing hosts to crawler nodes so one node owns each host's politeness state. Persist frontier checkpoints and recover ownership after node failure.
- Follow-up: identify the bottleneck when the entire site corpus must be recrawled within 24 hours; bandwidth and DNS resolution become primary constraints, motivating capacity planning and a dedicated DNS cache.

## Notes

- The key design decision is treating the frontier as a scheduler with two competing objectives, not a FIFO queue. A Bloom-filter false positive can skip a page entirely, so the error budget and an exact fallback for important domains must be explicit.
- Treat robots policy as host-level state keyed by scheme, host, and port, with an explicit expiration plus last-good and failure behavior. Keep that cache under the same ownership boundary as politeness state. For recrawls, conditional requests using validators such as ETag or Last-Modified can reduce bandwidth when the origin supports them.

## Preparation

- Draw the complete crawl loop and annotate where state must persist for restart.
- Work through a two-level frontier scheduler with three priority classes and several hosts, including robots.txt delays.
- Calculate the bandwidth and fetch-rate budget for a 24-hour recrawl, then explain how DNS caching, sharding, and backpressure interact.

SYSTEM DESIGN | ? min

# Large-Scale Interactive Map Visualization

Frequency: single | Last asked: 2026-08-29 | Stage: onsite-system-design | Roles: SWE | Tags: frontend, location, caching, scaling, sse, hard
Original: 1point3acres.com/.../interactive-map-100m-datapoints

*Design an interactive Google Maps visualization for more than 100 million data points. Use aggregated heatmaps at coarse zoom levels and individual pins at the final zoom, while addressing rendering technology, caching, live updates, client threading, failure recovery, and hardware/network limits.*

## Requirements

- Visualize more than 100 million geospatial data points on top of Google Maps.
- At broad zoom levels, aggregate points into a heatmap; at the final zoom level, render individual data pins.
- Map GIS data and aggregation resolution to map zoom level.
- Accept live updates over an SSE API and define the data-refresh frequency.
- Specify caching and fallback behavior across zoom levels, including how stale or missing data is handled.
- Account for client hardware, memory, threading, and network limits.
- Defend the rendering stack: SVG vs Canvas vs WebGL, and the trade-offs among libraries such as Chart.js, deck.gl, and D3.
- Explain the render cycle, client-worker model, shared-memory buffering, update path, and recovery behavior when live data becomes unavailable.

## Notes

- The opening prompt may contain almost no requirements beyond the visualization goal. Drive clarification one question at a time and connect each requested capability to a concrete need; the interviewer may challenge every requirement with "why do you need this?"
- Expect constraints to change during the discussion, including irrelevant additions intended to test prioritization. State assumptions, defend them, and make the impact of each change explicit instead of silently redrawing the architecture.
- A canonical end-to-end shape: pre-aggregate points into a zoom-level tile pyramid on the server so the client never receives the raw dataset; render heatmap tiles as textures and final-zoom pins with WebGL instancing; decode and diff incoming SSE updates in a web worker so the main thread only receives ready-to-draw buffers; cache tiles keyed by tile-and-zoom with fallback to the nearest coarser zoom when a tile is missing or stale; and on stream loss, reconnect from a checkpoint so missed updates replay instead of forcing a full reload.

## Preparation

- Sketch the zoom-dependent data path from raw GIS points to heatmap tiles and final-zoom pins.
- Compare SVG, Canvas, and WebGL for the render cycle, then justify one stack under client hardware and network constraints.
- Walk through caching with zoom fallback, live SSE refresh, worker/shared-memory coordination, and failure recovery as one end-to-end flow.

SYSTEM DESIGN | 60 min

# ML System Design (Recsys / Chatbot / Image Classifier)

Frequency: high | Last asked: 2026-10-02 | Stage: onsite-system-design, tech-screen | Roles: MLE, SWE, RS | Tags: system-design, retrieval, rag, scaling
Original: 1point3acres.com/.../ml-system-design-recsys-chatbot-classifier

*MLE / AI-ML L4-L5 loops include one ML System Design round (~60 min). Prompts span recommender, chatbot, image classifier, video moderation, fraud detection. Interviewers probe pipeline structure, feature engineering, training/serving split, cold start, A/B testing, and latency vs accuracy trade-offs.*

## Requirements

### Confirmed prompts

- Real-time recommendation system (e.g. for an e-commerce, social, or YouTube-like video platform).
- Mobile app: photograph a leaf, classify it into one of K species — retrieval + ranking architecture.
- Chatbot grounded in mixed structured + unstructured data — retrieval-augmented generation, citation, fallbacks.
- Short-video duplicate detection (anti-duplicate at billions-of-videos scale).
- NLP clustering / category expansion pipeline — explain data preparation, labeling strategy, model choice, evaluation, how the approach extends to new categories, and how the system is served.
- Internal-document search agent — design an agent over company documents, then reduce cost and latency, scale it to 100,000 users, and defend the agent framework.
- Music recommendation system — design the recommender, compare trade-offs and alternatives, then extend it into an LLM-powered recommendation agent.
- Watch-next recommendation — the classic "what to watch next" video recommender; expect the interviewer to go deep on modeling details rather than stay at the architecture level.

### The interviewer drives the discussion across these axes

- Functional requirements: traffic profile, latency SLO, freshness requirement.
- Data: training data sources, labeling, cold-start strategy.
- Pipeline: feature store, offline training, online serving, near-line updates.
- Model: baseline (LR / GBDT) → neural; two-tower retrieval + ranker stack for recsys.
- Scaling: sharding, embedding compression / quantization, ANN / LSH retrieval.
- Online: A/B testing, holdback, counterfactual eval, monitoring (drift, fairness, fallback).
- Cold start: content-based embedding, collaborative filtering, contextual bandits — give multiple options.
- Failure modes: stale features, training-serving skew, prediction drift, human-in-the-loop calibration after appeals.
- Latency vs accuracy: cache, distilled student model, near-line inference vs online.

## Notes

- Candidates who only said "collaborative filtering" for cold start were flagged shallow. Always offer ≥3 strategies and trade them off.
- Mention monitoring + iteration explicitly: interviewers down-rate candidates who design only the happy path without discussing post-launch.

- For the leaf classifier, the expected architecture is two-stage: vision encoder → embedding → ANN retrieval → re-ranker; don't jump straight to a 1k-way softmax.
- For the chatbot, RAG with citations is table stakes; bonus for discussing structured-data routing (SQL query vs vector search) and fallback policies.
- For the short-video dedup task, expect deep follow-ups on ANN cost vs accuracy, hot-key handling, embedding dimensionality vs latency.
- The NLP clustering variant rewards a complete ML lifecycle rather than model trivia: connect labels and evaluation to deployment, then close with serving and category expansion.
- The watch-next variant can be interrogated at formula level: be ready to write down the training loss you would use (and why), and the optimizer update rule, not just name them. A candidate who stayed at block-diagram depth did not pass.

## Preparation

- Whiteboard the canonical video-recommendation pipeline (candidate generation → two-tower retrieval → lightweight ranker → heavy ranker → re-ranking) twice cold; do the same for ad ranking and image search. Watching a walkthrough is not enough — redraw without the reference.
- Read "Designing Machine Learning Systems" by Chip Huyen, chapters on data and serving.
- Prepare a generic 6-section template: requirements → data → features → model → serving → monitoring. Drill walking through it in 50 minutes on any prompt.
- For each domain (recsys / chatbot / vision), prepare 1 quantitative example: target QPS, model size, latency budget.
- Practice the cold-start sub-question with a script: "Three approaches, here's when each wins, here's how I'd combine them."

**BEHAVIORAL | 45 min**

# Behavioral Round (Googliness Themes)

Frequency: very high | Last asked: 2026-10-16 | Stage: phone-screen, tech-screen, onsite-bq, onsite-coding | Roles: SWE, MLE, RS, INFRA-ENG, ALL-ROLES | Tags: behavioral, culture-fit
Original: 1point3acres.com/.../bq-round-google-style

*Standalone 45-minute BQ round used in nearly every Google interview loop (NG, intern, L4/L5/L6). Interviewers cycle through 6–10 stock prompts, take detailed notes, and aggressively follow up on the situation, action, and impact. The recurring themes are conflict, collaboration with different perspectives, ownership, dealing with ambiguity, and feedback.*

## Problem

- A single 45-minute slot dedicated to behavioral. Interviewers typically open with 5 minutes of self-introduction + 1 BQ, then 30 minutes of rapid-fire questions, then 10 minutes of candidate questions. Some loops keep BQ inside a coding slot (15 min BQ + 30 min coding).

## Format variations

- Some BQ slots stay shallow with only a few tell me about a time prompts, while others run 8–10 questions in a single 45-minute slot.
- A 45-minute variant can spend the entire slot drilling into one project. Know its trade-offs, rejected alternatives, performance limits, and your personal decisions well enough to sustain a full-depth conversation.
- Some R1 behavioral interviews begin directly with questions, without a self-introduction.
- Some new-grad loops fold behavioral questions into every round instead of running a standalone slot: each of four interviews combines a short behavioral segment (team projects, collaboration) with a coding problem.
- Some loops front-load BQ before coding (you must do well to unlock the next round); others interleave.

## Confirmed prompt families

### Conflict & feedback

- Tell me about a time you had a conflict with a teammate / manager and how you resolved it.
- A time you received critical feedback and what you did with it.
- A time you gave feedback to someone and how they reacted.
- A teammate is failing to deliver and the team is suffering — what do you do?
- Someone claims more credit than they deserve — how do you handle it?

### Ownership & impact

- Most challenging project you have led / contributed to; what was your specific impact.
- A cross-functional project you led or shaped deeply; explain how you aligned the participating organizations.
- A time you proactively found and removed a system performance bottleneck.
- A time you went beyond the requirements or stepped up to support the team unprompted.
- A mistake you made and what you learned.
- A time you missed a goal you set and what you learned.
- A time you made a risky decision; explain the trade-off and outcome.
- Your manager assigns a task that appears exceptionally difficult — how do you respond?

## Collaboration & inclusiveness

- Working with someone whose background or working style is dramatically different from yours.
- How you onboarded a new team member; what you took away from being onboarded yourself.
- A time you mentored someone: how you diagnosed what support they needed, adjusted your approach, and measured progress.
- A time someone helped you, and a time you helped someone else.
- A time you supported inclusiveness at work.
- A time you brought together different perspectives that led to a better design.
- What role do you naturally play on a team, and which behavioral traits define that role?

## Ambiguity & decisions

- A time you handled a constantly-changing project / shifting requirements.
- A time you operated with limited information; what you did.
- A time you had multiple competing deadlines; how you prioritized.
- A scenario question: "Tomorrow you start a new project, but you notice a bug in another team's project — what do you do?" (followed up with: "now the buggy project is yours, but the two projects have no dependency — does that change anything?").

## Recent additions

- Difficult customer or stakeholder; what you did and how you measured the outcome.
- Simplifying a complex process or system.
- Quickly learning a new technology to complete a task.
- Delivering under a tight deadline.
- A project you are most proud of and why.
- Taking ownership of a problem outside your formal responsibility.
- Why Google, and which aspect of Google's culture attracts you.
- AI ethics and responsible use.
- Testing mindset prompt: given a newly built image-detection app, explain how you would test it before launch.
- Product iteration prompt: describe a time you used user feedback to improve or build a feature.
- How you would keep yourself excited after joining.
- A teammate takes all the credit in a team status update.
- You have too much work at once; how do you prioritize.
- You propose an idea and others disagree.
- You work at a company with poor work-life balance and team atmosphere; what do you do, and what changes if you are the leader.

## Late-2026 additions

- If you could improve yourself right now, which direction would you pick, and how would it help your future work at Google?
- A past leader or manager who had a strong, lasting influence on you — how did you apply those learnings later?

## Two people disagree on an approach — how do you handle it?

- A disagreement with another team (not just a teammate) — how did you resolve it?
- A junior teammate you lead does not accept your direction — what do you do?

- A time your perception of a teammate changed from negative to positive.
- What organizational or company values are you looking for in your next role?
- A successful project; collaboration / project management; conflicts and trade-offs, typically anchored on a project deep dive.

### Long-tail / closing

- What would you do differently if you could revisit one of your projects?
- Most rewarding project of your life and why.

### What are the three most important attributes of a good team for you?

### What would Google as a large company improve on?

- What is your greatest achievement outside of work? (occasionally surfaces — clarify with the interviewer, it is not a misunderstanding)

### Product disagreement and technical debt

- Describe a serious disagreement with a Product Manager over product direction or implementation. How did you communicate and reach agreement?
- With an urgent deadline and technical debt hurting development efficiency, how do you balance delivery against code quality?

### Pitfalls

- Several current interviewers explicitly decompose STAR into separate sub-questions; answer in Situation / Task / Action / Result order even when the prompt is conversational.
- Interviewers will dig into every story: "Why was that action necessary?", "What measurable impact did it have?", "What would you do differently looking back?". Be ready with the second-layer details for every story.
- Director-level and senior interviewers tend to ask scenario / hypothetical questions; same-level peers tend to follow the script more closely.
- Treat the BQ as a hard signal, not a warmup.
- Pacing matters across shallow and 8–10-question formats; keep STAR responses under 3 minutes each and be ready for either style.
- "What is your achievement outside of work?" caught at least one candidate off guard — prepare one personal story (hobby project, volunteer work, athletic milestone) to avoid silence.
- At L5, mentoring and late requirement changes are explicit leadership probes. Prepare enough decision detail to explain how you influenced others, not only how you completed the work yourself.
- Follow-ups can revisit the causes of a difficult situation, not just the response: for example, why competing urgent tasks arose and whether earlier coordination could have prevented the conflict. A mentoring story can also be challenged when it covers technical help but omits career support.
- In loops that fold BQ into every round, keep two or three compact stories ready to drop into any round's opening minutes.
- Less-common L5 prompts (a teammate you came to see positively, the company values you want next) are hard to improvise; have a short, honest answer prepared for each.

### Follow-ups

- "Tell me about a time you have to compromise, and how will you act differently after it. Are you satisfied with the result? Did you do anything to make it happen later?"
- "Imagine you're part of the Google Photos team, and your feature detects smiling faces in photos. How will you verify the correctness of the detection?"

- "Describe a great tech leader you've ever seen, and do you want to become him/her? How do you measure your way becoming such a great tech leader"
- For competing urgent tasks, explain why the prioritization worked, the customer impact, and why the conflict was not identified earlier.
- For a mid-project direction change, explain how you verified the change with users and evaluated its success beyond relying on the Product Manager's judgment.
- If a team already gets along well, how would you make it more innovative?
- For helping someone join a team, distinguish technical onboarding from support for that person's career development.

### How would you handle a colleague taking credit for your work?

- How do you manage your time and coordinate across several tasks with different priorities?

### Why Google, and what will you do if this attempt does not succeed?

- If you were a manager, what would you expect from your own manager, and how would you practice it yourself

### Preparation

- Prepare 6–8 STAR stories that, across the set, hit every theme above (conflict, ownership, ambiguity, feedback, inclusiveness, mistake, scope-up, prioritization). Each story must be reusable for at least two prompts.
- For every story, write down quantitative impact + a one-sentence "what I'd do differently".
- Drill scenario prompts (the "tomorrow you find a bug…" style) with a 2-minute timer.
- Practice with a peer using only follow-up questions: "why", "what specifically did you say", "what was the measurable impact".
- Skim the Google Re:Work resources on Googliness (curiosity, comfort with ambiguity, helping others) and align story framing to those traits.

**OTHER ROUND | ? min**

# CPU Architecture and Simulator Design Deep Dive

Frequency: single | Last asked: 2026-07-24 | Stage: onsite-deep-dive | Roles: PERF-ENG | Tags: cpu-architecture, cpp, cs-fundamentals, technical-deep-dive, hard
Original: 1point3acres.com/.../cpu-architecture-simulator-deep-dive

*A virtual English-language CPU Architect loop combines an assembly-code test, a broad microarchitecture oral, and C++ simulator design. Core topics include branch prediction, MOESI, caches, MSHRs, VIPT, TLB/page walks, ROB behavior, simulation models, and simulator optimization.*

## Requirements

- The virtual English-language loop uses three technical rounds followed by a behavioral conversation with the hiring manager.
- Complete an assembly-code test.
- Explain modern CPU microarchitecture topics, including branch prediction, MOESI coherence, caches, MSHRs, VIPT, TLBs, page walks, and reorder buffers.
- Compare simulator time-advancement strategies (cycle-stepped vs. event-driven) separately from fidelity levels (cycle-accurate vs. cycle-approximate).
- Explain how to optimize a simulator without losing the fidelity required by its use case.
- Be ready to design a branch-predictor simulator module and justify its interfaces, state, update timing, and accuracy/performance trade-offs.

## Notes

- The questioning is difficult but interactive; hints are available when the reasoning is headed in a productive direction.
- Superficial definitions are insufficient. Each mechanism should connect to pipeline behavior, correctness, or measurable performance impact.

## Preparation

- Draw a modern out-of-order pipeline from memory and trace a branch miss, cache miss, TLB miss, and coherence transition through it.
- Implement a small branch-predictor simulator in C++ with deterministic traces and accuracy metrics.
- Build a two-axis comparison: cycle-stepped versus event-driven scheduling on one axis, cycle-accurate versus cycle-approximate fidelity on the other; evaluate speed, event ordering, debugging complexity, and accuracy.
- Practice reading short assembly sequences and explaining dependencies, stalls, speculation, and memory effects aloud.

OTHER ROUND | ? min

# Research Paper Presentation and Defense (DeepMind RS)

Frequency: single | Last asked: 2026-08-13 | Stage: onsite-research | Roles: RS | Tags: presentation, critique, deep-dive, vlm, hard
Original: 1point3acres.com/.../deepmind-rs-paper-presentation-defense

*Present one of your papers, then defend the motivation, design choices, assumptions, alternatives, limitations, and next research steps. For multimodal/VLM work, expect close scrutiny of training and post-training depth and whether the contribution is more distinctive than evaluation or benchmark assembly.*

## Requirements

- The round contains no coding. Present one of your own papers, then handle a fast sequence of research-defense questions:
- Why was this problem worth pursuing, and why was the chosen framing appropriate?
- Which alternative design could replace the one in the paper, and what would change?

## What happens if a central assumption does not hold?

## Why was each important design decision defensible?

- What is the most credible next research direction?

## Notes

- Motivation and judgment carry more weight than reciting method details. Evaluation- or benchmark-heavy work is vulnerable when the training and post-training story is shallow. When a question cannot be answered conclusively, expose a defensible line of reasoning instead of bluffing.

## Preparation

- Reduce the paper to a concise narrative covering motivation, hypothesis, decisive design choices, evidence, and limitations.
- Build an alternative-design table with credible rejected options, the trade-off behind each rejection, and the evidence that could reverse the decision.
- Rehearse an adversarial defense in which every assumption is challenged and every answer ends with a falsifiable next experiment.

OTHER ROUND | 60 min

# Google Hiring Assessment (GHA / Work Styles)

Frequency: high | Last asked: 2026-09-03 | Stage: oa | Roles: SWE, MLE, RS, INFRA-ENG, ALL-ROLES | Tags: behavioral, culture-fit
Original: 1point3acres.com/.../google-hiring-assessment-gha

*Non-coding online assessment sent by Google recruiting after a resume pass. Current variants use forced-choice work-style triples plus a situational judgement section on policy, ethics, confidentiality, and collaboration consistency.*

## Requirements

### Part 1 — Work Styles inventory / forced-choice triples

- Current common format: each prompt shows three workplace statements; choose one as Most like me, then choose one of the remaining two as Least like me.
- Earlier variants used single-statement Likert choices (Strongly Disagree / Disagree / Agree / Strongly Agree); the same consistency signal still applies.
- Coverage: ambiguity tolerance, collaboration, thoroughness, adaptability, communication, decisiveness, self-motivation, problem solving, leadership.
- The forced-choice triples often contain three positive traits, so the question is about your dominant working style rather than finding a single obviously correct answer.
- Common work-style clusters: reliable delivery / timeline ownership, collaboration across teams, careful reflective communication, structured problem solving, security / policy mindset, comfort with ambiguity.

### Part 2 — Situational Judgement (~10–15 items)

- Each item: a workplace scenario + 4 actions. Pick MOST likely and LEAST likely to do (cannot pick the same one for both).
- Recurring scenario families: gift / conflict of interest, confidential information leakage (laptop on vacation, coffee shop call, friend borrowing work laptop), policy vs deadline, manager unavailable, helping a colleague being treated unkindly, competitor offer, accepting a referral that benefits a friend.

### September 2026 three-part variant

- Complete forced-choice work-style triples, a separate Strongly Agree / Strongly Disagree section, and MOST / LEAST likely workplace scenarios in the same assessment. There is no coding.
- The agreement section includes process improvement, adaptability, learning from mistakes, feedback, teamwork, and resilience. Pay attention to absolute terms such as only, all, completely, and regardless.

## Examples

### Likert items to mark as "Strongly Agree"

- I quickly adjust my pre-existing work plans to meet changes in project goals.
- I complete my tasks effectively despite changing requirements.
- I am able to succeed in ambiguous work situations.
- I share resources with my colleagues to help them complete their work.
- I identify potential risks before I decide on a course of action.
- I keep detailed notes about the decisions made in meetings.

## Likert items to mark as "Strongly Disagree"

- I can overlook small, but important, errors in my work.
- When deciding between projects, I usually pick the project that will allow me to avoid working with others.
- I write reports with an emphasis on speed over accuracy.
- I require clear instructions at work to be effective.
- I cannot progress on a project when I don't have all the information I need to complete a task.

## Sample SJT scenarios

- A client offers a gift card after a project completes; company policy forbids gifts.
- You discover a coworker was mistakenly credited for your work right before annual review.
- A friend at a struggling competitor asks for confidential customer information.
- A new PM is being mocked by teammates for not catching up technically.
- Your manager calls to discuss confidential matters while you are in a coffee shop.

## Additional workplace scenarios

- A sales-system error awards you a colleague's credit, which would let you meet your annual target. Decide how to handle the mistaken credit.
- Your friend's startup is one of the software vendors under consideration, while policy requires an objective evaluation. Decide who should make the purchase decision.
- Sensitive employee data must stay on company devices, but the required software on your work laptop fails before tomorrow's deadline. Decide how to handle the delay without moving the data to a personal laptop.

## Notes

- The Likert column order is Strongly Disagree → Disagree → Agree → Strongly Agree (Strongly Agree is the rightmost, not the first). Several candidates have reported submitting reversed answers by accident and being immediately flagged.
- Submissions are final per question; you cannot go back to edit.
- Total time is 30–60 minutes; pacing varies, but completing slowly does not appear to penalize.
- Most candidates pass within 24–72 hours; passing the GHA does not guarantee continuing — some candidates still see resume rejection arrive after a pass.
- The 2026 forced-choice ranking format is now the main pattern in recent samples; consistency is still the dominant signal.
- Candidates commonly pass within two days after submission, then wait for recruiter or hiring-manager routing.

## Preparation

- Pre-commit to a coherent "work persona" before opening the test: strong ownership, collaborative, methodical, comfortable with ambiguity. Apply it uniformly across the ~50 inventory items.
- For the SJT, default to: follow policy, escalate to manager, decline gifts, never share confidential data, never use a competitor's code, ask before accepting external offers/help.
- Re-read every statement before submitting — reverse-worded items ("I avoid…", "I cannot…") are easy to flip.
- Triple-check the column order before answering the first question.

OTHER ROUND | 60 min

# ML Fundamentals Deep Dive (AI/ML & MLE Roles)

Frequency: high | Last asked: 2026-09-12 | Stage: onsite-deep-dive, phone-screen, tech-screen, onsite-research | Roles: MLE, RS, SWE, RE | Tags: ml-knowledge, transformer, rag, retrieval, deep-dive
Original: 1point3acres.com/.../ml-fundamentals-deep-dive

*One round in every MLE / AI/ML loop is a 45-60 minute domain interview: 10 min resume walkthrough, then rapid-fire ML fundamentals, often around the candidate's chosen track (LLM / NLP / CV / Recsys). Interviewers go deep on why each concept works, not just definitions.*

## Requirements

- The round is conducted as a Q&A oral exam plus light whiteboard derivations. Candidates pick a track when scheduling; the interviewer tailors questions to that track but always anchors to ML fundamentals.

## Core ML fundamentals (asked across most loops)

- Bias–Variance tradeoff; how to detect and mitigate each.
- Overfitting / underfitting; concrete remedies.
- Logistic regression: loss, optimization, decision boundary.
- Gradient descent variants: batch / mini-batch / SGD / momentum / Adam — when to use which.
- Cross-entropy: derivation, why preferred over MSE for classification.
- L1 vs L2 regularization: difference, why L1 produces sparsity (geometric explanation), why L2 is smoother.
- Forward / backward propagation: walk through chain rule, gradient flow, parameter update.
- Bayesian vs frequentist; likelihood; perplexity.
- Why Transformers beat RNNs; complexity of attention vs recurrent step; cost of autoregressive decoding.

## LLM / NLP track

- L4 add-on AI/ML rounds can concentrate on post-training, fine-tuning, prompt engineering, and base Transformer mechanics.
- Fine-tuning methods (full / LoRA / adapters / prefix); pros and cons of each.
- Inference: KV cache, top-p / top-k / nucleus / temperature sampling.
- MoE architectures.
- Tokenization methods compared (BPE / WordPiece / SentencePiece).
- Generative AI: autoencoder vs VAE vs VQ-VAE.
- Using an LLM to power product features (e.g. LLM for recommendations) — discuss when retrieval vs prompting wins.

## Recsys track

- Cold-start mitigations: content-based embeddings, collaborative filtering, contextual bandits — give multiple options, not just one.
- Two-tower retrieval vs ranking trade-offs.
- Online vs offline evaluation; counterfactual eval.
- LSH / ANN for retrieval at scale; embedding dimension vs latency.
- Human-in-the-loop calibration for moderation false positives.

## CV track

- ViT vs CNN; when each wins.
- Vector DBs for similarity search; product quantization.
- Quantization and pruning for production-scale serving.

### Post-training and probabilistic-modeling depth

- Explain GRPO and pipeline parallelism, including where they fit into an LLM training stack.
- Define KL divergence, perplexity, and importance sampling, then connect each concept to how and why it is used in modern LLM training.
- Derive the VAE objective: ELBO, the KL term, and the reparameterization trick; justify why each component is needed.
- Compare post-training methods by the model stage or problem they target.
- Identify concrete reward-hacking behaviors and explain how a post-training pipeline can prevent or detect them.

### Project-dive sub-section (~15 min)

- Pick one project from the resume; walk through problem framing, modeling choices, why other options were ruled out, results, limitations.
- Interviewer fishes for honesty about what didn't work and what you'd do differently — same Googliness signal as the BQ round.

### Notes

- The interviewer expects you to defend mechanism, not just terminology. "L1 gives sparsity" is not enough — you must show why the L1 ball corners snap weights to 0.
- Be ready to derive: softmax + cross-entropy gradient, attention forward pass, gradient flow through a residual block — on the whiteboard.
- The interviewer may not know your sub-track deeply. One candidate observed the interviewer keyword-matching against a script rather than engaging with niche ideas. Mention the standard answer first, then the more interesting modern variant — don't lead with niche papers.
- ML rounds in the L4 AI/ML loop tend to focus on project deep-dive over textbook fundamentals; L5/L6 lean more on fundamentals + architecture trade-offs.
- Cross-checking: research-scientist (RS) loops add 2–5 dedicated research rounds (talk + 1-on-1 deep-dives); MLE loops collapse research into this single ML round.
- In the PhD new-graduate AI/ML path, domain questions can align closely with the interviewer's own specialty. Interviewers may provide hints or explain a concept after the candidate reaches a gap in understanding.

### Preparation

- Print and drill the 3 Blue 1 Brown deep learning + linear algebra series for geometric intuition (L1/L2 ball, gradient field).
- Write out attention forward + softmax + cross-entropy gradients by hand 3+ times until automatic.
- For LLM track, read the Anthropic and OpenAI fine-tuning / KV-cache / sampling docs; memorize the trade-off table for top-p vs top-k vs temperature.
- For Recsys track, study the TIGER / two-tower / FAISS literature; have one paragraph ready for cold-start that combines collaborative filter + content embeddings.
- For CV track, drill ViT vs CNN trade-offs and one quantization paper at the level of detail you'd answer on a whiteboard.
- Have 1–2 personal projects ready for deep dive; rehearse explaining why you picked the approach over 2 alternatives.